

Claude Code — Guide des bonnes pratiques

Maîtrise des coûts, architecture mémoire, Skills & harness de développement guidé par l'IA

Édition VS Code (extension Claude Code)

Auteur : Ali DHIBI Fractional CTO & Strategic Advisor — AI & LLM Expert (Granite, LangGraph) — Cloud Architecture & Cybersecurity Site : <https://alidhibi.me>

Sommaire

- 1. CLI vs extension VS Code
- 2. Le problème `/usage` et le suivi de conso
- 3. Principes d'architecture : mémoire externalisée, pré-filtrage sémantique, discipline de session
- 4. Maîtriser la taille du contexte
- 5. Limiter le recours aux sous-agents
- 6. Prompts précis = sessions courtes
- 7. Stopper les boucles d'auto-correction
- 8. Outillage MCP
- 9. Choisir le bon modèle par tâche
- 10. Monitoring et alertes
- 11. Skills (Agent Skills)
- 12. Commands vs Sous-agents vs Skills
- 13. Le harness de développement guidé par l'IA
- 14. Le cycle Explore → Plan → Implement → Verify
- 15. Fichiers types prêts à committer
- 16. Checklist de mise en place
- 17. Résumé opérationnel
- 18. À propos de l'auteur

1. CLI vs extension VS Code

Tout ce qui vit dans le dossier `.claude/` (agents, settings, MCP, commands, skills) est **identique** entre la CLI et l'extension VS Code. Les seules différences concernent les **flags de lancement** (`claude --model`, `--max-turns`, etc.), qui **n'existent pas** dans l'interface de l'extension : tout passe alors par les **slash commands** et les **fichiers de configuration**.

Mécanisme	CLI (<code>claude ...</code>)	Extension VS Code
Réduire le contexte	<code>/compact</code>	<code>/compact</code> dans le chat
Repartir de zéro	<code>/clear</code>	<code>/clear</code> dans le chat
Voir le coût	<code>/cost</code>	<code>/cost</code> dans le chat
Changer de modèle	<code>/model</code> ou <code>--model</code>	<code>/model</code> dans le chat
Sous-agents	<code>.claude/agents/*.md</code>	Identique (fichiers versionnés)
Skills	<code>.claude/skills/*/SKILL.md</code>	Identique (fichiers versionnés)

Mécanisme	CLI (<code>claude ...</code>)	Extension VS Code
Limiter les tours	<code>--max-turns</code> OU <code>maxTurns</code>	<code>maxTurns</code> en config uniquement
Variables d'environnement	<code>export shell</code>	<code>settings.json</code> ou terminal
Suivi de consommation	<code>ccusage</code> , <code>/cost</code>	Identique
Plafonds de dépense	Console Anthropic	Identique

Règle d'or : investis dans les fichiers `.claude/` — ils sont partagés par toute l'équipe et fonctionnent partout.

2. Le problème `/usage` et le suivi de conso

La commande `/usage` affiche les **quotas d'abonnement** (fenêtres de 5 h des plans Pro / Max). Elle ne fonctionne **que si tu es authentifié avec un compte d'abonnement Claude**.

Le message `Usage tracking is only available for Claude AI subscribers.` signifie que tu es authentifié **via une clé API (facturation Console / pay-per-use)**. C'est normal : sur l'API, il n'y a pas de quota de plan à afficher.

Situation	Outil de suivi recommandé
Facturation API / Console	<code>/cost</code> + <code>ccusage</code> + dashboard Console (le standard équipe)
Abonnement Pro / Max individuel	<code>/usage</code> (après <code>/login</code> avec le compte d'abonnement)

Vérifier sa méthode d'authentification :

```
# Dans Le chat Claude Code
/status      # affiche la méthode d'auth active (API key vs subscription)
/login      # relance le flux d'authentification

# Dans Le terminal intégré VS Code
echo $ANTHROPIC_API_KEY      # si rempli → mode API (pas de /usage)
```

Pour une équipe facturée à l'usage, restez sur l'API et adoptez `ccusage` + **Console** comme standard de suivi. Le tracking Console est de toute façon supérieur à `/usage` pour le pilotage budgétaire.

3. Principes d'architecture : mémoire externalisée, pré-filtrage sémantique, discipline de session

Ces trois principes forment le socle d'un usage économique et fiable de Claude sur des missions longues. Ils déplacent la continuité **hors** de la fenêtre de conversation.

3.1 Mémoire externalisée, pas conversationnelle

Le modèle ne « se souvient » pas d'une conversation qui s'allonge indéfiniment. On s'appuie plutôt sur :

- Un **fichier d'instructions permanent** (`CLAUDE.md`) qui joue le rôle de **manuel opératoire**.
- Un ensemble de **fichiers mémoire structurés** (`session_state.md` , notes de décisions, ADR) qui **persistent entre les sessions**.

Le cycle devient : **relire un état documenté** → **travailler** → **écrire ses conclusions dans la mémoire** → **repartir de zéro sur un contexte propre**. On gagne à la fois en **coût** (pas d'historique qui gonfle) et en **fiabilité** (l'état est explicite, versionné, revu).

Approche	Coût	Fiabilité sur session longue
Contexte conversationnel qui gonfle	Croît sans cesse	Se dégrade (dérive, oublis)
Mémoire externalisée + reset	Stable et bas	Élevée (état écrit noir sur blanc)

Exemple de prompt :

```
"Avant de commencer, lis CLAUDE.md et session_state.md. À la fin de la tâche, écris tes conclusions et décisions dans session_state.md (fichiers modifiés, choix d'archi, points bloquants), puis on fera /clear."
```

3.2 Pré-filtrage sémantique avant toute lecture

Plutôt que de charger un fichier ou un document entier dans le contexte, on **indexe le corpus en amont dans une base vectorielle**, et l'agent **interroge cette base** pour ne récupérer que les **extraits réellement pertinents**.

- Résultat concret : on divise souvent **par 10** le volume de tokens chargés inutilement.
- On ne perd rien en pertinence — c'est l'inverse : on **cible mieux**.

Mise en œuvre concrète dans Claude Code / VS Code :

1. Indexer le corpus (codebase, docs, tickets, wiki) sous forme d'**embeddings** dans une base vectorielle (pgvector, Qdrant, LanceDB, etc.).
2. Exposer une **recherche sémantique via un serveur MCP** (voir section 8 et l'exemple `.mcp.json`).
3. Discipliner l'agent : **interroger la base d'abord**, ne lire les fichiers entiers qu'en dernier recours.

Sans pré-filtrage	Avec pré-filtrage sémantique
Lecture de fichiers/docs entiers	Récupération des seuls extraits utiles
Contexte lourd, coûteux, bruité	Contexte léger, ciblé, précis
Risque de rater l'info dans le bruit	Meilleure pertinence

Exemple de prompt :

```
"Avant de lire quoi que ce soit : interroge le serveur de recherche sémantique (mcp semantic-search) avec la requête 'validation des remboursements'. Ne charge que les extraits retournés. Ne lis un fichier entier que si un extrait est insuffisant, et dis-le explicitement."
```

3.3 Discipline de session

On découpe la mission en **phases séquentielles**, chacune dans **sa propre session**, avec un **reset de contexte** (`/clear`) entre chaque phase. La continuité ne repose plus sur « ce que le modèle a en tête », mais sur ce qui est **écrit noir sur blanc** dans la mémoire externe.

C'est ce qui évite **l'accumulation silencieuse de contexte** qui plombe à la fois le coût et la précision sur les sessions longues.

Boucle de discipline par phase :

1. `/clear` (contexte propre)
2. `"Lis CLAUDE.md + session_state.md"` (recharge l'état)
3. Travail de la phase (périmètre strict)
4. `"Mets à jour session_state.md"` (persiste l'état)
5. `/clear` → phase suivante

Synthèse des 3 principes : la mémoire vit dans des **fichiers** (pas dans le chat), on **filtre sémantiquement** avant de lire, et on **reset le contexte** entre les phases. Coût maîtrisé, fiabilité accrue.

4. Maîtriser la taille du contexte

Chaque token envoyé **et** reçu est facturé, et le contexte grossit à chaque tour. C'est le principal levier de coût.

- Évite les fichiers larges inutiles (`CLAUDE.md` verbeux, imports massifs, lockfiles).
- Utilise `/compact` régulièrement pour résumer l'historique en cours de session.
- Utilise `/clear` quand tu changes complètement de tâche (plus économique que `/compact`).
- Préfère des **sessions courtes et ciblées** plutôt qu'une longue session fourre-tout.

Spécificités VS Code :

- Ferme les onglets inutiles : selon ta configuration, le contexte de l'éditeur peut être ajouté automatiquement.
- Utilise les références explicites `@fichier` plutôt que de laisser Claude lire un dossier entier.
- Surveille l'indicateur de remplissage de la fenêtre de contexte dans le panneau de chat.

Exemples de prompts :

```
# Compacter en ciblant ce qu'on veut garder
"/compact en gardant uniquement les décisions d'architecture et la liste
des fichiers modifiés. Oublie les logs de tests intermédiaires."

# Cibler le contexte au lieu de tout charger
"Lis uniquement @src/auth/login.ts et @src/auth/session.ts. N'explore pas
le reste du dossier auth pour l'instant."
```

5. Limiter le recours aux sous-agents

Chaque sous-agent est spawné via le **Task tool** : Claude crée une instance fraîche avec son propre contexte, ses outils restreints et son modèle. Quand il termine, **seul un résumé remonte** à la session principale.

L'impact coût est **linéaire** : 10 agents parallèles consomment le quota 10× plus vite. Un agent A qui appelle B qui appelle C provoque une **explosion exponentielle**.

Antipatterns à éviter :

- **Prompts de sous-agents trop longs** : viser 100–300 mots max. Au-delà, tu ne spécialises pas, tu recharges le même contexte dans chaque fenêtre.
- **Paralléliser des tâches dépendantes** : si B a besoin de l'output de A, exécute en séquentiel.
- **Laisser Claude décider seul** du fan-out : définis explicitement quand orchestrer.

Règle d'arbitrage pour l'équipe :

Type de tâche	Profil d'agent	Modèle
Lecture / exploration	Explore (read-only)	Haiku
Analyse d'architecture	Plan (pas d'écriture)	Opus
Implémentation	general (tous outils)	Sonnet
Tests / vérification	Verify (borné)	Haiku

Exemples de prompts :

```
# Déléguer explicitement à un sous-agent
"Utilise l'agent codebase-explorer pour localiser où sont gérés les
webhooks Stripe. Ne modifie rien, rapporte juste les fichiers concernés."

# Forcer le séquentiel sur des tâches dépendantes
"Étape 1 : agent architect pour planifier la migration. STOP, attends ma
validation. Étape 2 seulement après validation : on implémente."

# Interdire le fan-out
"Traite ces 3 fichiers toi-même, en séquentiel. Ne spawne pas de
sous-agents."
```

6. Prompts précis = sessions courtes

Un prompt vague pousse Claude à explorer et itérer, donc à générer plus de tokens pour converger.

- Définis clairement le **périmètre** : fichiers concernés, ce qu'on **ne touche pas**, format attendu.
- Utilise un **checkpoint** (`session_state.md`) pour reprendre sans tout recontextualiser.
- Dans VS Code, **sélectionne le code concerné** dans l'éditeur avant de poser ta question.

Exemples de prompts :

```
# Prompt vague (à éviter)
❌ "Améliore la gestion des erreurs"

# Prompt précis (à privilégier)
✅ "Dans @src/api/users.ts uniquement, remplace les `throw new Error(...)`
par notre classe AppError (voir @src/errors/AppError.ts). Ne touche pas
aux signatures de fonctions ni aux tests. Format attendu : un diff."

# Reprendre depuis un checkpoint
"Lis session_state.md et reprends l'étape en cours. Périmètre strict aux
fichiers listés. STOP et rapporte si bloqué après 2 essais."
```

7. Stopper les boucles d'auto-correction

Si Claude échoue et réessaie en boucle (tests qui cassent, erreurs de lint), les tokens s'accumulent silencieusement.

Garde-fous, par ordre de fiabilité dans l'extension :

Garde-fou	Portée	Fiabilité extension
Condition d'arrêt dans le prompt	Universelle	Excellente
<code>maxTurns</code> en config d'agent	Sous-agents	Bonne
Interruption manuelle (Stop)	Session courante	Manuelle

Le flag CLI `--max-turns` et `--no-auto-fix` **n'existent pas** dans l'UI de l'extension. Le contrôle passe par `maxTurns` (config d'agent) et, surtout, par une **condition d'arrêt explicite dans chaque prompt**.

Exemples de prompts :

- ✗ "Fix the failing tests"
- ✓ "Fix the failing tests. If you can't fix them in 2 attempts, stop and output: BLOCKED: [reason]. Don't retry further."
- ✓ "Corrige l'erreur de lint dans `@src/utils/date.ts`. Une seule passe. Si l'erreur persiste, STOP et explique pourquoi sans réessayer."

8. Outillage MCP

Les tools MCP (filesystem, terminal, recherche sémantique, etc.) génèrent des `tool_result` qui **restent dans l'historique** et grossissent vite.

- Limite les lectures de fichiers larges via MCP.
- Restreins les `tools` autorisés par agent.
- **Scope** toujours les serveurs MCP (ex. `./src`, jamais la racine du repo).
- Audite régulièrement quels serveurs MCP sont actifs.
- **Bon usage de MCP** : exposer une **recherche sémantique** (base vectorielle) pour le pré-filtrage de la section 3.2, au lieu de charger des documents entiers.

Exemples de prompts :

- "Avant de lire via le serveur MCP filesystem, liste d'abord les fichiers correspondants avec Glob et ne charge que ceux de moins de 200 lignes."
- "Interroge d'abord le serveur MCP semantic-search ; ne lis un fichier entier que si les extraits sont insuffisants."

9. Choisir le bon modèle par tâche

La stratégie recommandée repose sur trois niveaux : **Opus** pour le raisonnement complexe, **Sonnet** pour l'essentiel du travail, **Haiku** pour les tâches simples.

Switch manuel dans la session (slash commands, fonctionne dans l'extension) :

- `/model haiku` # vérification, lint, lecture simple
- `/model sonnet` # tâche standard (≈ 80 % des cas)

```
/model opus      # architecture, debugging profond, décisions critiques
```

Le mode `opusplan` (Opus pour planifier, bascule sur Sonnet pour coder) se sélectionne via `/model` .

Matrice modèle / tâche à diffuser :

Tâche	Modèle conseillé
Exploration codebase, lecture	Haiku
Refacto standard, ajout de feature	Sonnet
Revue de PR, génération de tests	Sonnet
Architecture, debugging complexe	Opus (phase plan)
Migration infra, décisions critiques	Opus

Exemples de prompts :

```
# Tâche simple en Haiku
"/model haiku"
"Liste tous les TODO et FIXME du dossier @src et regroupe-les par fichier."

# Réflexion en Opus, puis redescendre pour coder
"/model opus"
"Conçois la stratégie de migration de la table users. Donne le plan, ne
code rien."
# ... après validation ...
"/model sonnet"
"Implémente l'étape 1 du plan validé."
```

10. Monitoring et alertes

ccusage — voir où partent les tokens (à lancer dans le terminal intégré) :

```
npx ccusage@latest daily --breakdown    # conso par jour, ventilée par modèle
npx ccusage@latest blocks --live        # fenêtres de facturation 5 h en temps réel
npx ccusage@latest monthly              # vue mensuelle
npx ccusage@latest daily --since 20260501 --until 20260601  # analyser un pic
```

/cost dans la session : coût et tokens consommés de la session en cours.

Console Anthropic : console.anthropic.com → **Usage** (graphiques) et **Settings > Limits** (plafond mensuel + alertes email à 50 %, 80 %, 100 %).

Anti-bruit de fond : positionner `DISABLE_NON_ESSENTIAL_MODEL_CALLS=1` dans `settings.json` .

Indicateurs à suivre par sprint :

Indicateur	Source
Tokens / session par dev	<code>ccusage daily --breakdown</code>

Indicateur	Source
Sessions Opus non justifiées	Logs / Console
Fan-out de sous-agents (20+)	Observation des sessions
Cascades de compaction (longues sessions)	Observation des sessions
Boucles de retry sur contexte resoumis	Observation des sessions

Désigner un **budget owner** chargé du suivi hebdomadaire.

11. Skills (Agent Skills)

Un **Skill** est un dossier qui package une expertise réutilisable : des instructions (`SKILL.md`), et éventuellement des **scripts** et des **ressources** (templates, schémas). Claude charge automatiquement un skill quand sa `description` correspond à la tâche en cours.

Pourquoi les Skills sont un levier de coût (progressive disclosure) :

- Seules les **métadonnées** (`name` + `description`) restent en permanence dans le contexte — très léger.
- Le **corps du** `SKILL.md` n'est chargé que lorsque le skill est réellement déclenché.
- Les **scripts** s'exécutent de façon **déterministe** : pas de tokens générés pour de la logique répétitive.

Structure d'un skill :

```
.claude/skills/  
└─ nom-du-skill/  
  │ SKILL.md           # obligatoire : frontmatter + instructions  
  │ scripts/          # optionnel : scripts exécutables (déterministes)  
  │   └─ generate.mjs  
  │ references/        # optionnel : templates, schémas, exemples  
  │   └─ template.ts
```

Portées disponibles :

Portée	Emplacement	Usage
Projet	<code>.claude/skills/</code>	Versionné, partagé par l'équipe
Personnel	<code>~/.claude/skills/</code>	Tes skills perso, non versionnés

Bonnes pratiques :

- La `description` doit dire **quand** utiliser le skill (c'est elle qui déclenche le chargement).
- Garde le `SKILL.md` concis ; déporte les détails volumineux dans `references/`.
- Mets la logique répétitive et déterministe dans `scripts/`.

Déclencher un skill :

```
/skills           # liste les skills chargés (selon version)  
"Utilise le skill create-endpoint pour ajouter une route GET /invoices."
```


12. Commands vs Sous-agents vs Skills

Critère	Slash Command	Sous-agent	Skill
Déclenchement	Explicite (<code>/commande</code>)	Délégation (Task tool)	Auto (description) ou explicite
Contexte	Session courante	Nouvelle fenêtre isolée	Session courante (disclosure)
Coût de présence	Nul tant qu'inutilisé	Crée un contexte complet	Métadonnées seules au repos
Idéal pour	Workflow lancé par l'humain	Tâche lourde à isoler	Expertise réutilisable, scripts
Peut exécuter du code	Oui (via prompt)	Oui (selon tools)	Oui (scripts packagés)

Règle simple :

- **Command** = « je veux lancer ça à la demande » (ex. `/review-pr`).
- **Sous-agent** = « délègue cette grosse tâche dans un contexte séparé ».
- **Skill** = « applique cette procédure/expertise quand le sujet apparaît ».

13. Le harness de développement guidé par l'IA

Le **harness** est le cadre versionné qui structure la façon dont l'équipe travaille avec Claude. Il rend les sessions reproductibles, moins coûteuses et prévisibles.

Composants du harness (tous versionnés dans Git) :

Fichier / dossier	Rôle
<code>CLAUDE.md</code>	Mémoire projet concise (manuel opératoire)
<code>.claude/agents/</code>	Sous-agents spécialisés (model + maxTurns + tools restreints)
<code>.claude/commands/</code>	Slash commands réutilisables et éprouvées
<code>.claude/skills/</code>	Skills : expertise + scripts déterministes (disclosure)
<code>.mcp.json</code>	Serveurs MCP scopés (dont recherche sémantique)
<code>.claude/settings.json</code>	Variables d'env et permissions partagées
<code>session_state.md</code>	Mémoire externalisée / checkpoint des tâches longues

Pourquoi ça réduit les coûts : le contexte essentiel est dans `CLAUDE.md` , les comportements répétitifs dans les agents/commandes/skills, la mémoire dans des fichiers externes, et le modèle est toujours adapté. L'investissement initial se rembourse en quelques sprints.

Bonne pratique d'équipe : traiter le harness comme du code — revu en PR, amélioré collectivement, avec un mainteneur désigné.

14. Le cycle Explore → Plan → Implement → Verify

Phase	Agent / outil	Modèle	Écriture	Garde-fou
Explore	codebase-explorer	Haiku	Non	Read-only + pré-filtrage sémantique
Plan	architect	Opus	Non	Read-only + validation humaine
Implement	session general (+ skills)	Sonnet	Oui	Périmètre strict
Verify	test-runner	Haiku	Oui	maxTurns + STOP/BLOCKED

Reset de contexte (/clear) entre chaque phase ; l'état transite par session_state.md .

Exemples de prompts par phase :

```
# Explore
"Utilise l'agent codebase-explorer. Interroge d'abord semantic-search sur
'validation des paiements', puis rapporte les fichiers clés (chemin:ligne).
Ne modifie rien."

# Plan
"/model opus"
"Utilise l'agent architect pour planifier l'ajout d'un système de
remboursement. Read-only. Termine par PLAN PRÊT et attends ma validation."

# Implement (après validation, nouveau contexte)
"/clear"
"Lis session_state.md. Implémente l'étape 1 du plan. Périmètre :
@src/payments/refund.ts + son test. Utilise le skill create-endpoint."

# Verify
"Utilise l'agent test-runner. 2 tentatives max, puis STOP + BLOCKED.
Mets ensuite à jour session_state.md."
```

15. Fichiers types prêts à committer

Arborescence cible :

```
.
├─ CLAUDE.md
├─ session_state.md
├─ .mcp.json
├─ .claude/
│   ├── settings.json
│   └─ agents/
│       ├── architect.md
│       ├── codebase-explorer.md
│       ├── code-reviewer.md
│       └─ test-runner.md
├─ commands/
│   └─ review-pr.md
├─ skills/
│   └─ create-endpoint/
│       ├── SKILL.md
│       ├── scripts/
│       │   └─ scaffold.mjs
│       └─ references/
│           └─ endpoint.template.ts
```

CLAUDE.md

```
# Projet – Mémoire Claude Code (manuel opératoire)
# Garder ce fichier CONCIS. Chaque ligne est rechargée à chaque session = coût.

## Stack
- Langage / framework : <ex. TypeScript, Node 20, React 18>
- Package manager      : <ex. pnpm>
- Base de données      : <ex. PostgreSQL via Prisma>

## Commandes essentielles
- Install / Dev / Build / Test / Lint / Typecheck : pnpm <cmd>

## Conventions de code
- ESLint + Prettier imposés (ne pas contourner).
- Pas de `any` en TypeScript ; typer explicitement.
- Tests obligatoires pour toute nouvelle fonction métier.
- Commits : Conventional Commits (feat:, fix:, chore:...).
```

Mémoire & contexte (discipline)

- Au début de chaque phase : lire CLAUDE.md + session_state.md.
- À la fin : écrire décisions/état dans session_state.md, puis /clear.
- Pré-filtrer via le serveur MCP semantic-search avant de lire des fichiers.

Zones sensibles – NE PAS MODIFIER sans validation humaine

- src/auth/**, migrations/**, infra/**, .env*

Règles de travail avec Claude

- PLAN avant implémentation pour toute tâche non triviale.
- Périmètre strict aux fichiers mentionnés.
- Si bloqué après 2 tentatives : STOP + `BLOCKED: [raison]`.
- Haiku pour explorer, Sonnet pour coder, Opus pour l'archi.
- Utiliser le skill create-endpoint pour toute nouvelle route API.

.claude/settings.json

```
{
  "env": {
    "DISABLE_NON_ESSENTIAL_MODEL_CALLS": "1"
  },
  "permissions": {
    "allow": [
      "Read",
      "Grep",
      "Glob",
      "Bash(pnpm test:*)",
      "Bash(pnpm lint:*)",
      "Bash(pnpm typecheck:*)",
      "Bash(node .claude/skills/**)",
      "Bash(git status)",
      "Bash(git diff:*)"
    ],
    "deny": [
      "Read(./env)",
      "Read(./env.*)",
      "Read(**/secrets/**)",
      "Bash(rm -rf:*)",
      "Bash(git push:*)"
    ]
  }
}
```

.claude/agents/architect.md

```

---
name: architect
description: Phase PLAN. Conçoit l'architecture et le plan d'implémentation. Read-only, ne code jamais.
model: claude-opus-4-1
maxTurns: 10
tools: Read, Grep, Glob
---

Tu es un architecte logiciel senior. Ton rôle est de PLANIFIER, jamais d'implémenter.

Procédure :
1. Lis session_state.md, puis explore le code pertinent (Read/Grep/Glob ciblés).
2. Identifie les contraintes : dépendances, zones sensibles (cf. CLAUDE.md), risques.
3. Produis un PLAN structuré :
  - Objectif (1-2 phrases)
  - Fichiers à créer / modifier (chemins précis)
  - Fichiers à NE PAS toucher
  - Étapes ordonnées et atomiques
  - Stratégie de test / vérification
  - Risques et décisions humaines requises
4. Termine TOUJOURS par : "PLAN PRÊT – valider avant implémentation."

Contraintes : READ-ONLY, pas de code complet, pose tes questions si ambigu,
reste concis (une page max).

```

.claude/agents/codebase-explorer.md

```

---
name: codebase-explorer
description: Explore et cartographie le code pour répondre à une question. Read-only, économique.
model: claude-haiku-4-5
maxTurns: 8
tools: Read, Grep, Glob
---

Tu réponds à une question d'architecture au moindre coût.

Procédure :
1. Interroge la recherche sémantique / Grep / Glob pour CIBLER avant de lire.
2. Ne lis que les portions de fichiers pertinentes.
3. Synthèse : réponse directe, fichiers clés (chemin:ligne), flux si utile.

Contraintes : READ-ONLY, évite les fichiers larges entiers, sois bref.

```

.claude/agents/code-reviewer.md

```

---
name: code-reviewer
description: Revue de code ciblée sur un fichier, un diff ou une PR. Read-only.
model: claude-haiku-4-5
maxTurns: 6
tools: Read, Grep, Glob
---

Tu es un reviewer de code senior.

1. Lis UNIQUEMENT les fichiers ou le diff fournis.
2. Liste priorisée : Bloquant / Important / Mineur.
3. Chaque point : fichier:ligne + explication courte + suggestion.

Contraintes : aucune modification, aucune commande, stop dès l'analyse finie.
Si RAS : "RAS – code conforme".

```

.claude/agents/test-runner.md

```

---
name: test-runner
description: Lance les tests et rapporte les échecs. Corrige au maximum 2 fois.
model: claude-haiku-4-5
maxTurns: 5
tools: Bash, Read, Edit
---
1. Exécute `pnpm test`.
2. Si OK : "Tous les tests passent" et STOP.
3. Si échec : correction MINIMALE et ciblée, puis relance.
4. Après 2 tentatives échouées : STOP + "BLOCKED: [résumé + fichiers]".
   NE CONTINUE PAS au-delà de 2 tentatives.

Contraintes : code strictement nécessaire, ne jamais truquer les tests,
rester dans le périmètre de l'échec.

```

.claude/commands/review-pr.md

```

---
description: Lance une revue de code complète sur les changements en cours
---
1. Exécute `git diff --staged` (ou `git diff` si rien n'est staged).
2. Délègue l'analyse à l'agent code-reviewer.
3. Résultat groupé par sévérité (Bloquant / Important / Mineur).
4. Verdict : APPROUVÉ / À CORRIGER / BLOQUANT.
Lecture seule, aucune modification.

```

.claude/skills/create-endpoint/SKILL.md

```

---
name: create-endpoint
description: >
  Crée un nouvel endpoint API REST en respectant les conventions du projet
  (routing, validation, gestion d'erreurs, test). À utiliser dès que
  l'utilisateur demande d'ajouter une route, un endpoint ou un handler API.
---
# Skill : création d'endpoint API

## Quand l'utiliser
Toute demande d'ajout d'une route HTTP (GET/POST/PUT/DELETE) côté API.

## Procédure
1. Déduis : méthode HTTP, chemin, schéma d'entrée, schéma de sortie.
2. Génère le squelette via le script déterministe (pas de génération manuelle) :
  `node .claude/skills/create-endpoint/scripts/scaffold.mjs <method> <path>`
3. Adapte le fichier généré à la logique métier demandée.
4. Respecte : validation Zod, gestion d'erreurs via AppError, un test
  minimal (cas nominal + cas d'erreur).
5. Ne modifie jamais src/auth/** sans validation humaine.

## Sortie attendue
- 1 handler + 1 test + enregistrement de la route + résumé des fichiers.

## Détails
Voir references/endpoint.template.ts pour le squelette de référence.

```

.claude/skills/create-endpoint/scripts/scaffold.mjs

```

#!/usr/bin/env node
// Génère un squelette d'endpoint de façon déterministe (zéro token LLM).

```

```

// Usage: node scaffold.mjs <method> <routePath>
// Exemple: node scaffold.mjs GET /invoices

import { writeFileSync, mkdirSync, existsSync } from "node:fs";
import { dirname, join } from "node:path";

const [, , methodArg, routeArg] = process.argv;
if (!methodArg || !routeArg) {
  console.error("Usage: scaffold.mjs <method> <routePath>");
  process.exit(1);
}

const method = methodArg.toUpperCase();
const name = routeArg.replace(/^\//, "").replace(/[\V:]/g, "-") || "root";
const handlerPath = join("src", "api", `${name}.ts`);
const testPath = join("tests", "api", `${name}.test.ts`);

const handler = `import { z } from "zod";
import { AppError } from "../errors/AppError";

export const ${name}Schema = z.object({
  // TODO: définir le schéma d'entrée
});

export async function ${name}Handler(req: unknown) {
  const input = ${name}Schema.parse(req);
  // TODO: logique métier
  return { ok: true };
}
`;

const test = `import { describe, it, expect } from "vitest";
import { ${name}Handler } from "../../src/api/${name}";

describe(`${method} ${routeArg}`, () => {
  it("cas nominal", async () => {
    expect(true).toBe(true);
  });
  it("cas d'erreur", async () => {
    expect(true).toBe(true);
  });
});
`;

for (const [p, content] of [[handlerPath, handler], [testPath, test]]) {
  if (existsSync(p)) {
    console.error(`SKIP (existe déjà): ${p}`);
    continue;
  }
  mkdirSync(dirname(p), { recursive: true });
  writeFileSync(p, content);
  console.log(`CRÉÉ: ${p}`);
}

console.log(`\nÀ FAIRE: enregistrer la route ${method} ${routeArg} dans le routeur.`);

```

.claude/skills/create-endpoint/references/endpoint.template.ts

```

// Squelette de référence d'un endpoint conforme aux conventions du projet.
// Chargé à la demande par le skill (progressive disclosure).
import { z } from "zod";
import { AppError } from "../../src/errors/AppError";

export const InputSchema = z.object({
  // champs validés en entrée
});

export type Input = z.infer<typeof InputSchema>;

```

```
export async function handler(rawInput: unknown) {
  const input = InputSchema.parse(rawInput); // 400 si invalide
  if (!input) {
    throw new AppError("INVALID_INPUT", "Entrée invalide", 400);
  }
  // Logique métier...
  return { ok: true };
}
```

session_state.md

```
# Session State – <nom de la tâche>
# Mémoire externalisée / checkpoint de reprise.
# Dernière MAJ : <date> – Auteur : <dev>

## Objectif global
<2-3 phrases>

## État d'avancement
- [x] Étape réalisée
- [ ] Étape en cours : <description précise>
- [ ] Étape suivante

## Fichiers concernés


| Fichier             | État     | Note                       |
|---------------------|----------|----------------------------|
| src/auth/login.ts   | modifié  | logique OK, tests à écrire |
| src/auth/session.ts | à créer  |                            |
| tests/auth.test.ts  | en cours | 2 cas restants             |



## Hors périmètre (ne pas toucher)
- migrations/**
- src/payments/**

## Décisions prises
- <ex. JWT en cookie httpOnly (sécurité).>
- <ex. Refresh token en DB, table `sessions`.>

## Bloquants / en attente
- BLOCKED: <description> – décision humaine attendue ?

## Prochaine action à lancer
"/clear puis : lis session_state.md et implémente l'étape en cours,
périmètre limité aux fichiers listés. STOP si bloqué après 2 essais."
```

.mcp.json (avec recherche sémantique)

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "./src"]
    },
    "semantic-search": {
      "command": "npx",
      "args": ["-y", "<votre-serveur-mcp-de-recherche-vectorielle>"],
      "env": {
        "VECTOR_DB_URL": "<url-de-votre-base-vectorielle>",
        "EMBEDDINGS_MODEL": "<modele-d-embeddings>"
      }
    }
  }
}
```

```
}  
}
```

16. Checklist de mise en place

- ☐ Créer l'arborescence `.claude/` (agents, commands, skills) et committer.
- ☐ Adapter `CLAUDE.md` à la stack réelle (commandes, zones sensibles, discipline mémoire).
- ☐ Adapter les commandes `Bash(...)` de `settings.json` au package manager.
- ☐ Adapter le skill `create-endpoint` (script + template) au framework réel.
- ☐ Indexer le corpus dans une base vectorielle + brancher le serveur MCP `semantic-search`.
- ☐ Instaurer la discipline de session : `/clear` + relecture `session_state.md` entre phases.
- ☐ Vérifier l'auth (`/status`) ; standard de suivi : `ccusage` + Console.
- ☐ Poser un plafond de dépense dans la Console + alertes 50 / 80 / 100 %.
- ☐ Désigner un budget owner (revue `ccusage daily --breakdown hebdo`).
- ☐ Générer le PDF de ce guide et le partager au kickoff.

17. Résumé opérationnel

Levier	Action clé
Mémoire	Externalisée dans des fichiers (<code>CLAUDE.md</code> , <code>session_state.md</code>), pas le chat
Pré-filtrage	Base vectorielle + MCP semantic-search : ne charger que les extraits utiles
Discipline session	Phases séquentielles + <code>/clear</code> entre chaque, état écrit noir sur blanc
Contexte	<code>/compact</code> régulier, fichiers concis, références <code>@fichier</code> ciblées
Sous-agents	Configs YAML avec <code>model</code> + <code>maxTurns</code> + <code>tools</code> restreints
Skills	Procédures réutilisables + scripts déterministes (progressive disclosure)
Boucles	Condition d'arrêt explicite (<code>BLOCKED</code>) dans chaque prompt
Modèle	Haiku/Sonnet par défaut, Opus sur décision consciente
Budget	<code>ccusage</code> + <code>/cost</code> + plafonds Console + budget owner
Cycle	Explore (Haiku) → Plan (Opus, validé) → Implement (Sonnet) → Verify (Haiku borné)

18. À propos de moi

Ali DHIBI Directeur Technique visionnaire, je cumule plus de 20 ans d'expérience en ingénierie logicielle et en leadership technique. J'ai développé une solide expertise dans la conception d'architectures SaaS et On-Premise scalables, ainsi que dans la conduite de transformations digitales pour des marchés internationaux.

Je suis spécialisé dans l'IA souveraine (LLMs), les infrastructures Cloud-Native basées sur Kubernetes (K8s) et les pratiques DevSecOps. J'accompagne les organisations dans la définition et l'exécution de feuilles de route technologiques alignées sur les

objectifs de croissance, tout en garantissant les exigences de conformité et de sécurité, notamment en matière de RGPD et de cybersécurité.

Habitué à piloter des équipes pluridisciplinaires (développement, QA, DevOps et support) je veille à créer un environnement de collaboration efficace, orienté performance, innovation et qualité de service.

Site personnel : <https://alidhibi.me>

Ce document peut être partagé au sein de votre équipe. Pour toute question d'architecture IA, de mise en place de harness de développement guidé par l'IA, ou de stratégie LLM, contactez moi